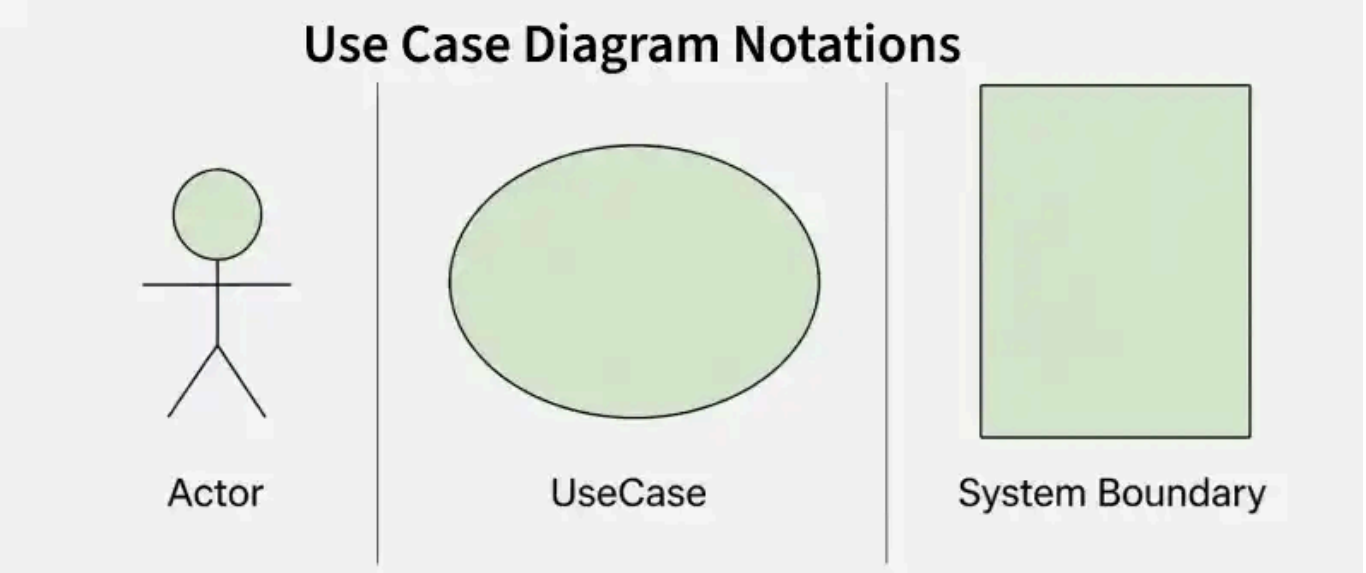
USE CASE DİYAGRAMLARI VE UML TEMELLERİ

1. UML NEDİR? (Neden Çizim Yapıyoruz?)

UML (Unified Modeling Language), yazılım sistemlerini görselleştirmek, tasarlamak ve dokümante etmek için kullanılan **standart bir çizim dilidir**.

> Benzetime gerek yok, doğrudan söyleyelim: UML, mühendislerin ortak konuştuğu görsel dildir. Tıpkı mimarların kat planı çizmesi gibi, yazılımcılar da UML çizer. Use Case Diyagramı, UML'in **12 farklı diyagram türünden biridir** ve **en üst seviye (en soyut) olanıdır.**



Use Case Diyagramı Ne İşe Yarar?

- Sistemin **kimler** tarafından (Actor) kullanıldığını,
- Bu kişilerin sistemde **hangi işlemleri** (Use Case) yapabildiğini,
- Bu işlemlerin birbiriyle **nasıl ilişkili** olduğunu gösterir.

> **Kritik Not:** Use Case diyagramı, **NASIL yapılacağını değil, NE yapılacağını** gösterir. Kod içermez, veritabanı içermez, altyapı içermez. Sadece "Kullanıcı burada şunu yapabilir" der.

2. USE CASE DİYAGRAMININ TEMEL YAPITAŞLARI

Bir Use Case diyagramında 3 temel öge vardır. Bunları asla unutma:

Öge	Şekli (Çizim Notasyonu)	Anlamı
Actor (Aktör)	Çizgi adam (stick figure) ikonu	Sistemle etkileşime geçen **dış varlık**. Genellikle insan, bazen harici sistem veya cihaz.

Öge	Şekli (Çizim Notasyonu)	Anlamı
Use Case (Kullanım Senaryosu)	Elips (oval) şekli	Sistemde gerçekleştirilen **bir işlev veya görev**.
Sistem Sınırı (System Boundary)	Dikdörtgen kutu	Use Case'lerin içinde yer aldığı sistemin **sınırları**. Actor'ler bu kutunun **dışındadır**.

3. İLİŞKİ TÜRLERİ (BAĞLANTILAR) – OYUN PROJESİ ÜZERİNDEN

Use Case'ler birbirinden bağımsız değildir. Aralarında 4 temel ilişki türü vardır. Bunları anlamak, diyagramı profesyonel yapar.

İlişki Türü	Çizgi Gösterimi	Ne Anlama Gelir?	Oyun Örneği (Karanlık Orman)
Association (İlişkilendirme)	Düz çizgi	Actor ile Use Case arasındaki **etkileşimi** gösterir. Hangi aktör hangi işlemi yapabilir?	"Oyuncu" actor'ü, "Yürü" Use Case'ine bağlanır.
Include (Dahil Etme)	--<>--> (kesikli ok)	Bir Use Case, **zorunlu olarak** başka bir Use Case'i çağırır. Ayrı çağrılamaz.	"Ateş Et" her zaman "Mermi Azalt"ı içerir. Mermi azalmadan ateş edemezsin.
Extend (Genişletme)	--<>--> (kesikli ok)	Bir Use Case, **opsiyonel olarak** başka bir Use Case'i çağırır. Sadece belirli koşullarda çalışır.	"Canavar Gör" Use Case'i, eğer oyuncu el fenerini açarsa "El Feneri Yak" ile genişletilir. Ama zorunlu değil.
Generalization (Genelleme)	İçi boş üçgen ok	Actor veya Use Case arasında **ebeveyn-çocuk** ilişkisidir. Çocuk, ebeveynin tüm özelliklerini miras alır.	"Premium Oyuncu" actor'ü, "Oyuncu" actor'ünü genelleştirir. Premium oyuncu her şeyi yapar, üstüne "Özel Kostüm" de giyer.

4. GERÇEK OYUN PROJESİ ÖRNEĞİ: "KARANLIK ORMAN" USE CASE DİYAGRAMI

Şimdi teorik bilgileri, az önce gereksinimlerini topladığımız hayatta kalma-korku oyununa uygulayalım.

Adım 1: Aktörleri (Actor) Belirleyelim

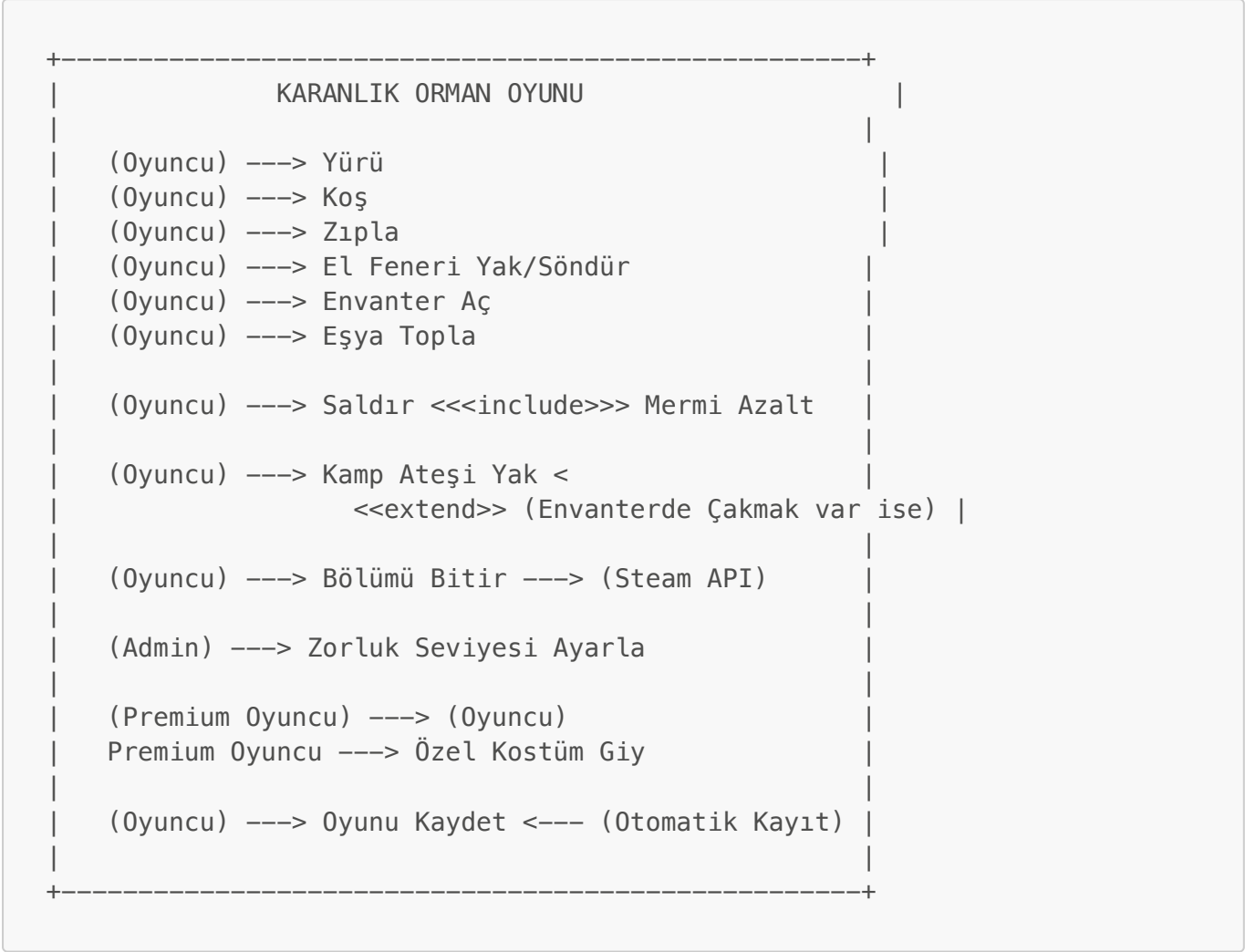
- **Ana Aktör:** Oyuncu (Player)
- **Yan Aktör:** Oyun Yöneticisi (Game Admin) – oyun içi ayarları uzaktan değiştirebilir.
- **Harici Sistem Aktörü:** Steam API – başarımları (achievement) kaydeder.

Adım 2: Use Case'leri Listele (Fonksiyonel Gereksinimlerden)

- Yürü, Koş, Zıpla, El Feneri Yak/Söndür, Envanter Aç, Eşya Topla, Kamp Ateşi Yak, Saldır, Canavar Kaçır, Bölümü Bitir, Oyunu Kaydet, Steam Başarımı Gönder.

Adım 3: İlişkileri Kur ve Sınırı Çiz

Diyagramı şöyle kurgularız:



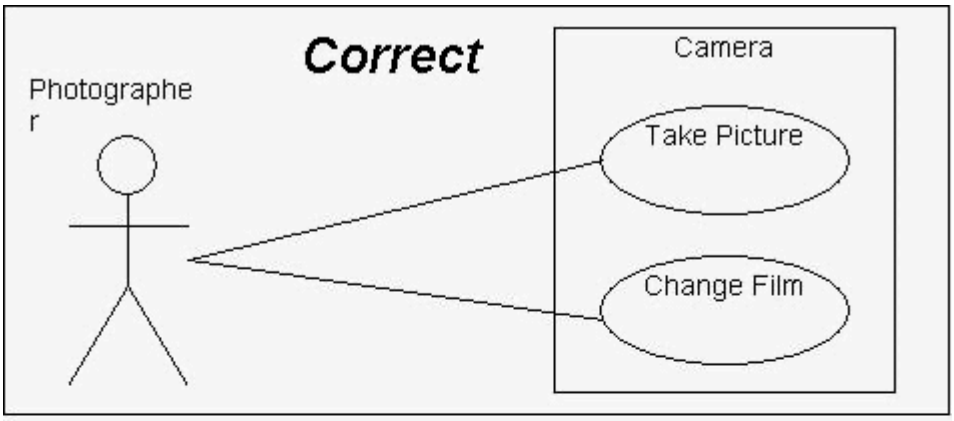
Adım 4: Bu Diyagram Bize Ne Söyler?

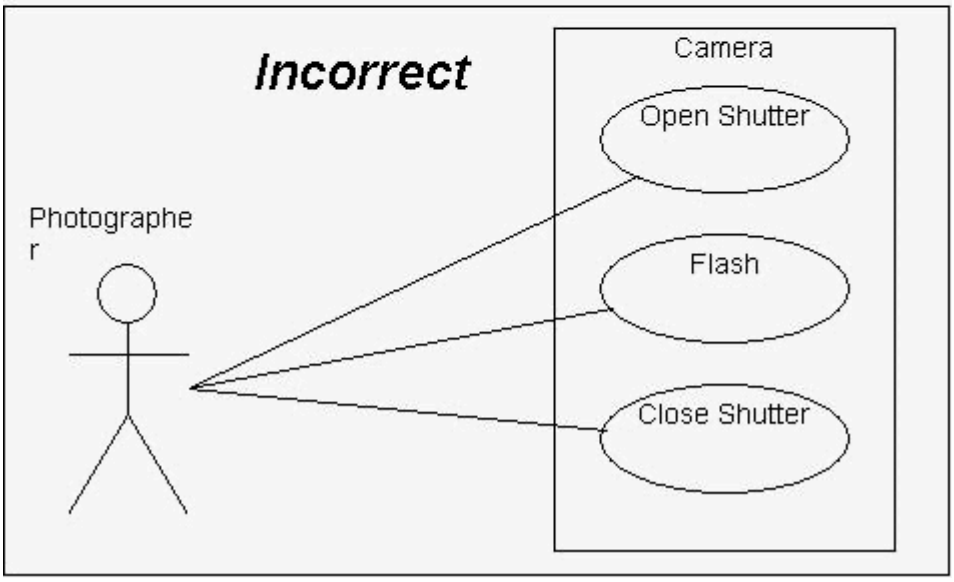
- **Include İlişkisi:** "Saldır" Use Case'i, "Mermi Azalt" olmadan çalışamaz. Yani kod yazarken önce mermi kontrol fonksiyonu çağrılmalıdır.
- **Extend İlişkisi:** "Kamp Ateşi Yak", sadece oyuncunun envanterinde çakmak varsa gerçekleşir. Bu opsiyonel bir durumdur, ana akışı bozmaz.
- **Generalization:** "Premium Oyuncu" normal oyuncunun yaptığı her şeyi yapabilir, üstüne ekstra özelliği vardır.
- **Actor-Use Case:** Steam API ile oyuncu etkileşmez, oyun arka planda başarımı gönderir. Bu yüzden Steam API harici sistem aktörüdür.

---

5. USE CASE DİYAGRAMI ÇİZERKEN SIK YAPILAN HATALAR

Hata	Doğrusu
Use Case'leri çok detaylı göstermek (örnek: "Veritabanına bağlan")	Use Case'ler işlevdir, teknik detay değildir.
Actor'leri sistem kutusunun içine koymak	Actor her zaman **dışarıdadır** (sistemle etkileşen dış varlıktır).
Include ve Extend'i karıştırmak	Include: Zorunlu, Extend: Opsiyonel
Her şeyi tek bir diyagrama sığdırmak	Büyük sistemlerde **birden fazla** Use Case diyagramı çizilir. Her modülün kendi diyagramı olabilir.
Use Case'leri isimlendirirken "crud" işlemleri (Ekle/Sil/Güncelle) yazmak	Bunlar çok alt seviyedir. "Siparişi Yönet" gibi daha üst seviye yaz.





Uygulanacak Adımlar;

1. Yeni Diyagram > UML > Use Case seç.
2. Actor aracını tıkla ve tuvala ekle (stick figure).
3. Use Case aracını tıkla (oval) ve ekle.
4. System Boundary aracını tıkla (dikdörtgen) ve tüm use case'leri içine al.
5. Actor'leri Use Case'lere düz çizgiyle bağla.
6. Include/Extend ilişkilerini kesikli oklarla belirle ve üzerine yaz (<>, <>).

7. Diyagramı PDF veya PNG olarak dışa aktar.

---

7. SİSTEM ANALİSTİNİN USE CASE DİYAGRAMI ÜZERİNDEKİ ROLÜ

- **Toplayıcı:** Gereksinimleri dinler, bunları Use Case'lere dönüştürür.
- **Sadeleştirici:** Karmaşık iş süreçlerini, birkaç oval ve çizgiyle özetler.
- **İletişimci:** Geliştiricilere "Bu oyunda neler var?" sorusunun cevabını tek bakışta sunar.
- **Doğrulayıcı:** Diyagramı müşteriyle / paydaşlarla birlikte inceler, "Evet, tüm istediklerim burada" onayını alır. Bu onay, ileride "Bunu istememiştim" tartışmalarını önler.

---

8. ARTILAR VE EKSİLER (DİYAGRAM YÖNETİMİ)

Artılar	Eksiler
Gereksinimleri görselleştirir, anlaşılır kılar.	Fazla detaylandırılırsa anlaşılmaz olur.
Paydaşlar arasında ortak dil oluşturur.	Sistemin "NASIL" çalıştığına dair hiçbir bilgi vermez.
Test senaryoları yazmak için birebirdir.	Dinamik davranışları (zamanlama, döngü) gösteremez.
Değişiklikler kolaydır (çizgi/oval ekle/çıkar).	Çok büyük sistemlerde 100+ oval çizilmez, modüllere bölünür.

---

9. ÖZET

- **UML** = Yazılımın görsel alfabesi.
- **Use Case Diyagramı** = Bu alfabenin ilk ve en anlaşılır harfi.
- **Actor** = Kim yapıyor? (Çizgi adam)
- **Use Case** = Ne yapıyor? (Oval)
- **Include** = Zorunlu bağımlılık (A yapılmadan B yapılmaz).
- **Extend** = Opsiyonel bağımlılık (A yapılırsa B de yapılabilir).
- **Generalization** = Ebeveyn-çocuk ilişkisi (Çocuk, ebeveynin yaptığı her şeyi yapar + fazlası).
- **Araçlar:** Draw.io (bireysel, ücretsiz), Lucidchart (ekip, profesyonel).
- - https://plantuml.com
- - https://www.drawio.com/docs/diagram-types/uml/use-case-diagrams/
-

E-Ticaret Sistemi Use Case Diyagramları

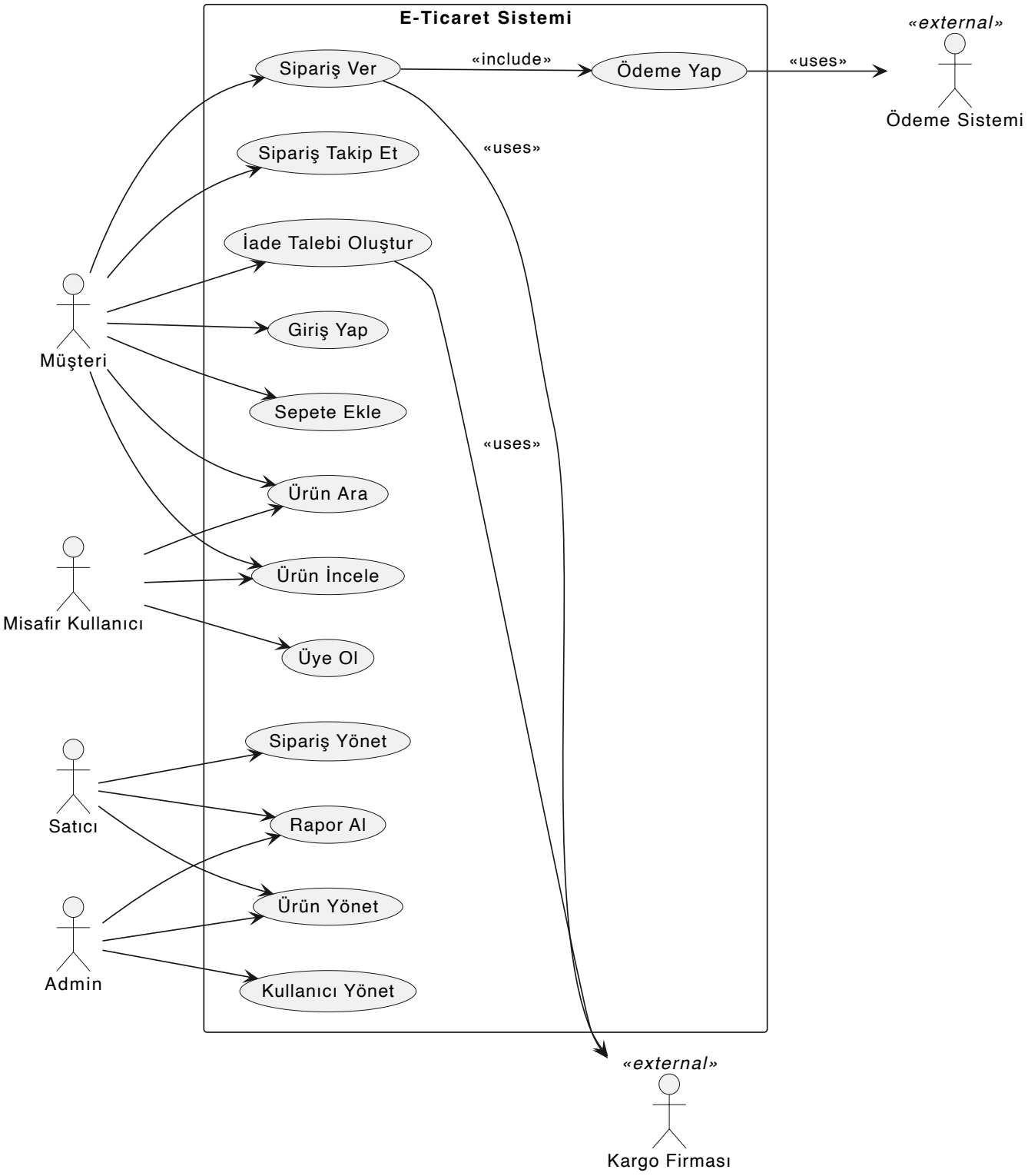
Aşağıda bir e-ticaret sistemi için **kapsamlı use case diyagramları** verilmiştir. Diyagramlar **PlantUML** formatında olup, doğrudan çizim araçlarına yapıştırılabilir.

---

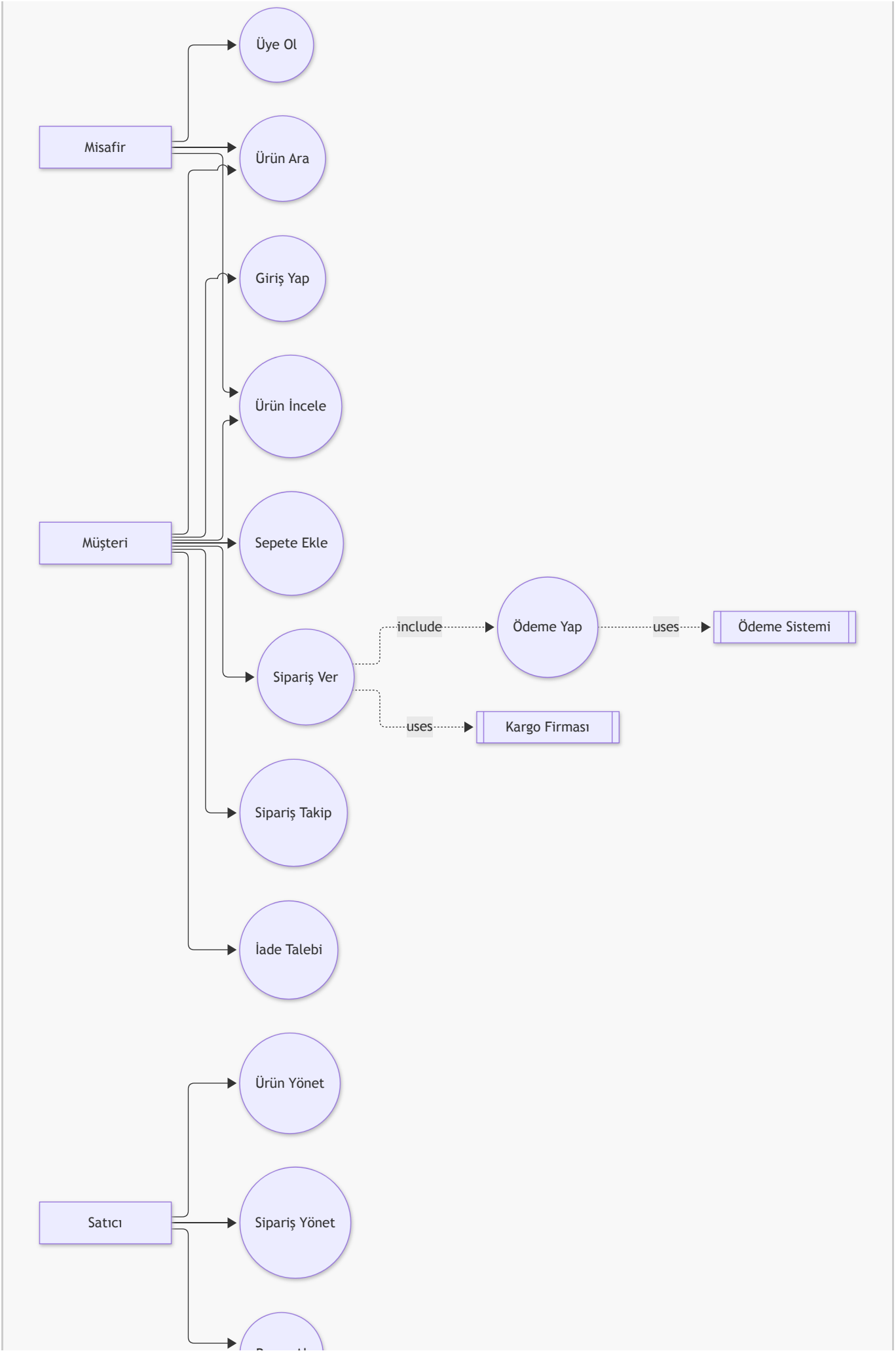
1. GENEL SİSTEM USE CASE DİYAGRAMI

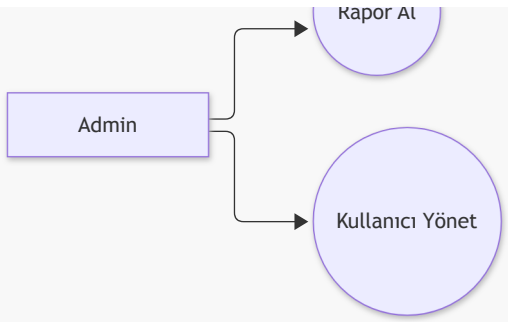
Tüm aktörleri ve ana use case'leri gösterir.

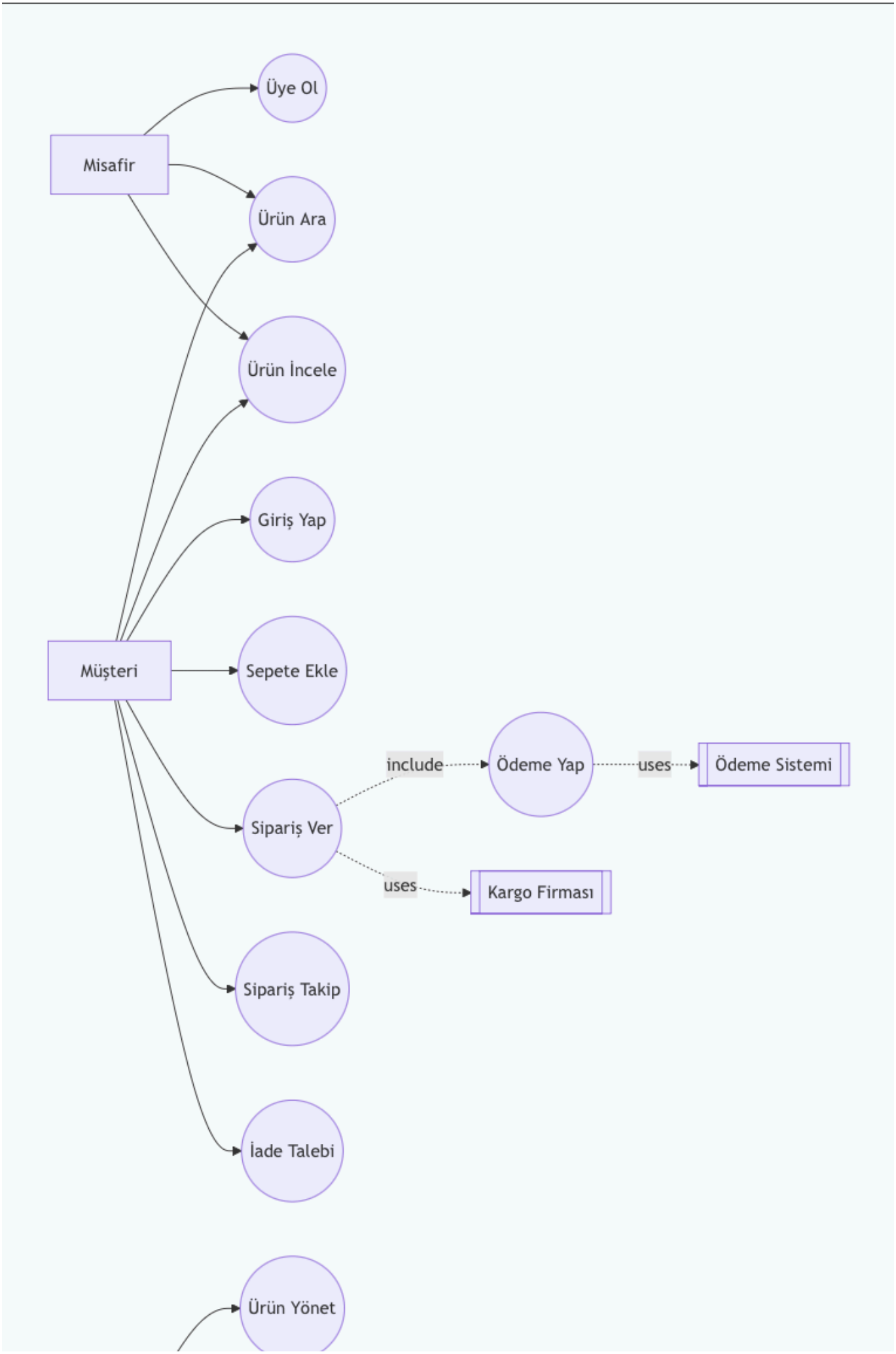
PlantUML

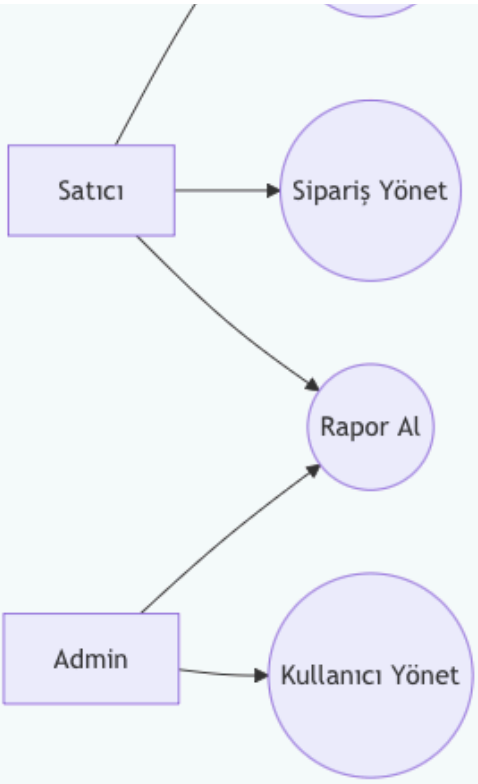


Mermaid


Misafir
Üye Ol
Ürün Ara
Müşteri
Giriş Yap
Ürün İncele
Sepete Ekle
Sipariş Ver
include
Ödeme Yap
uses
Ödeme Sistemi
uses
Kargo Firması
Sipariş Takip
İade Talebi
Satıcı
Ürün Yönet
Sipariş Yönet


Rapor Al
Admin
Kullanıcı Yönet

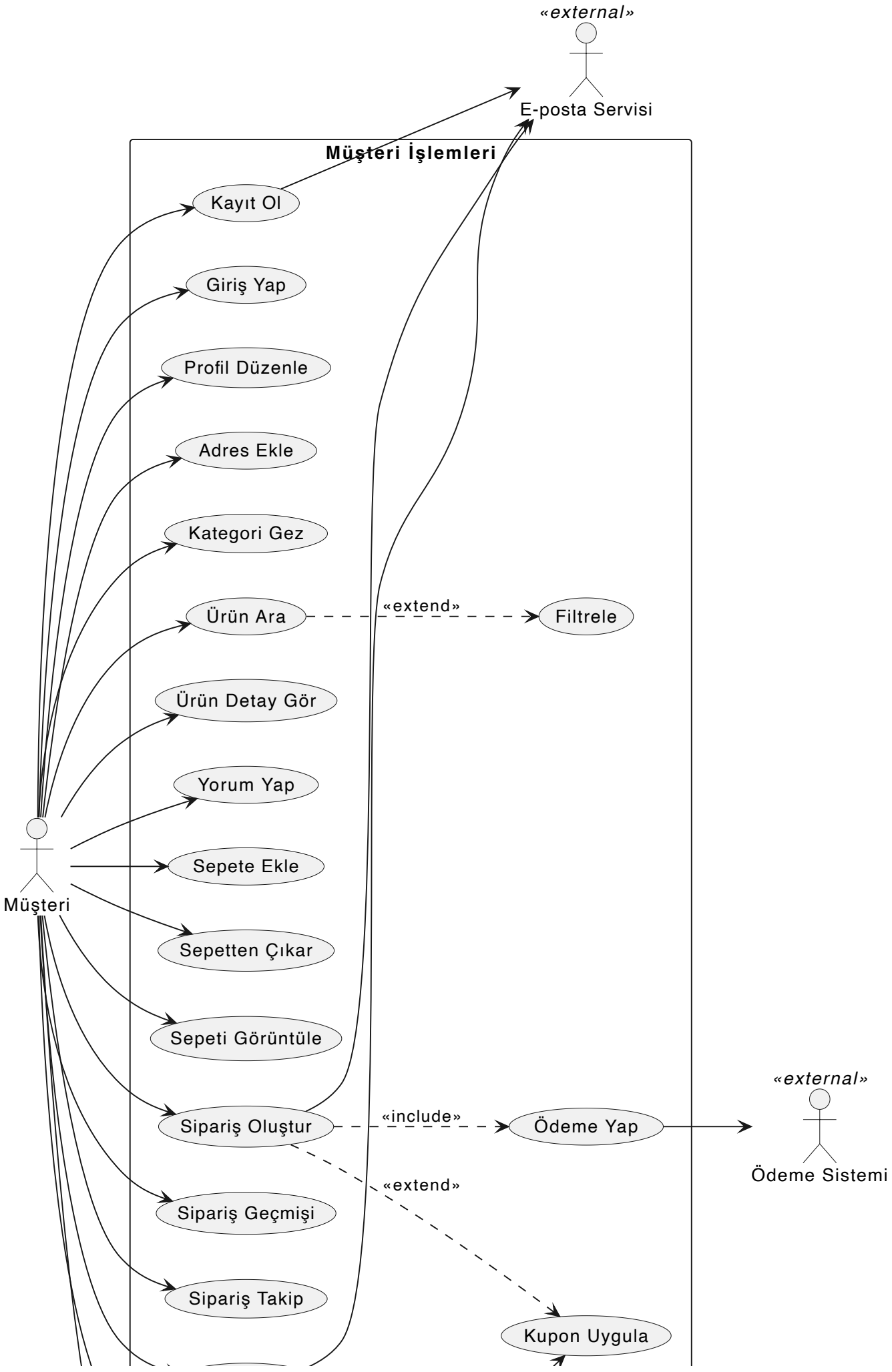

Misafir
Üye Ol
Ürün Ara
Ürün İncele
Müşteri
Giriş Yap
Sepete Ekle
Sipariş Ver
include
Ödeme Yap
uses
Ödeme Sistemi
uses
Kargo Firması
Sipariş Takip
İade Talebi
Ürün Yönet

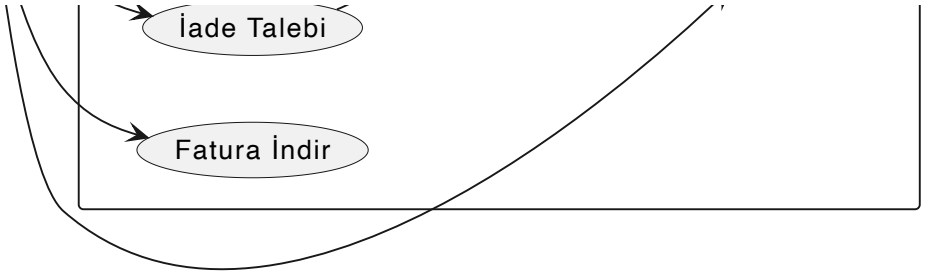


2. MÜŞTERİ (CUSTOMER) USE CASE DİYAGRAMI

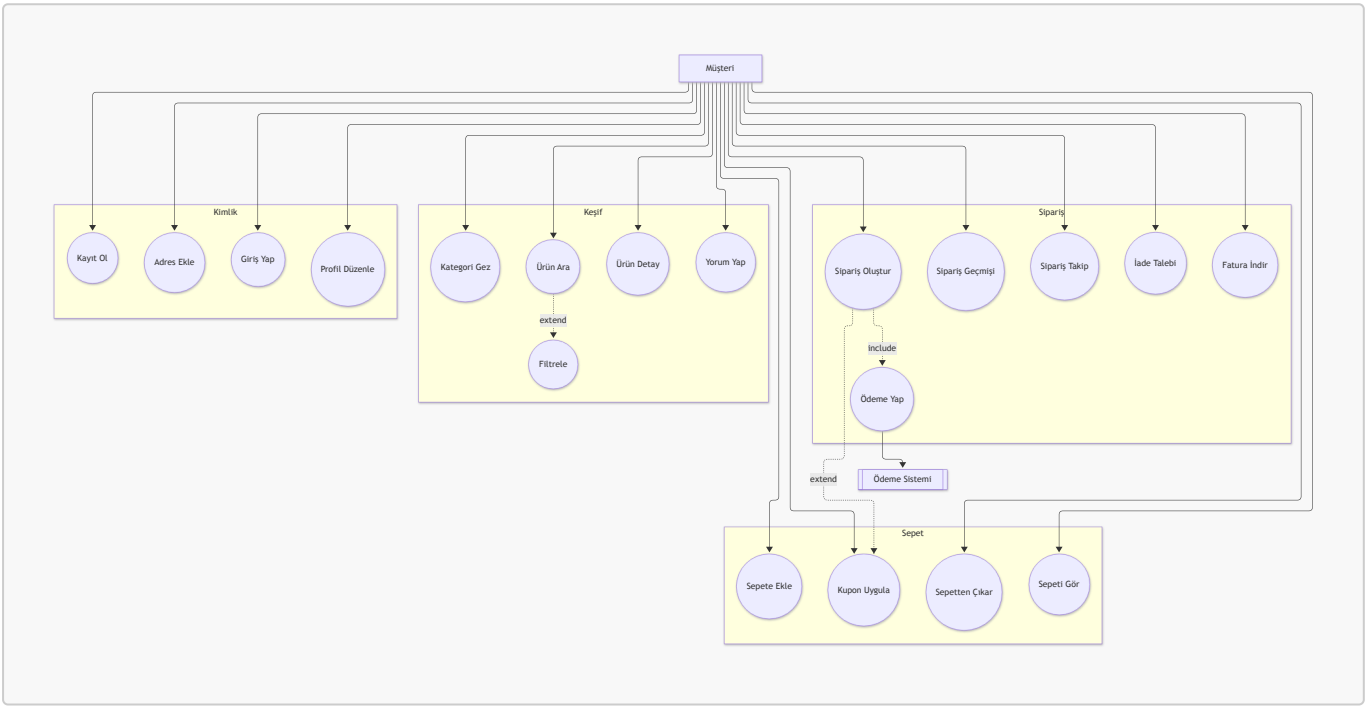
Müşterinin tüm etkileşimlerini detaylandırır.

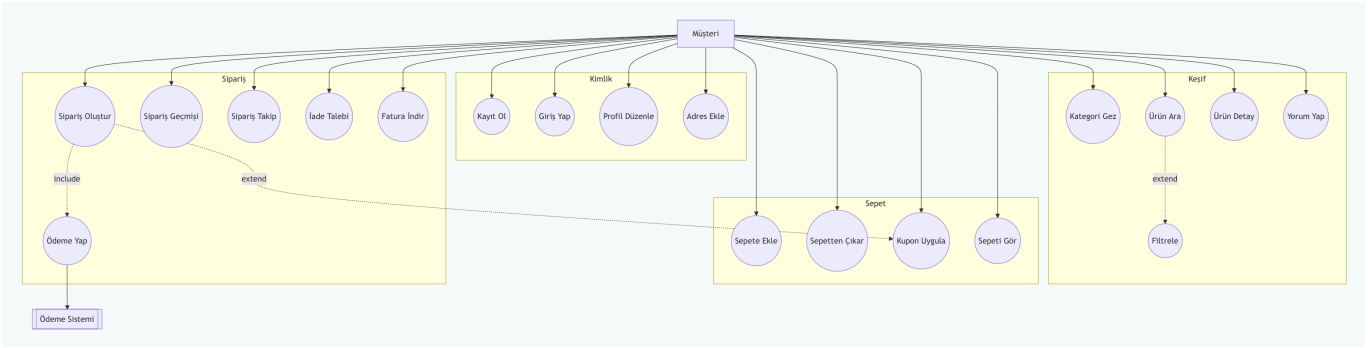
PlantUML


«external»
E-posta Servisi
Müşteri İşlemleri
Kayıt Ol
Giriş Yap
Profil Düzenle
Adres Ekle
Kategori Gez
Ürün Ara
«extend»
Filtrele
Ürün Detay Gör
Yorum Yap
Sepete Ekle
Sepetten Çıkar
Sepeti Görüntüle
Sipariş Oluştur
«include»
Ödeme Yap
«external»
Ödeme Sistemi
«extend»
Sipariş Geçmişi
Sipariş Takip
Kupon Uygula
Müşteri



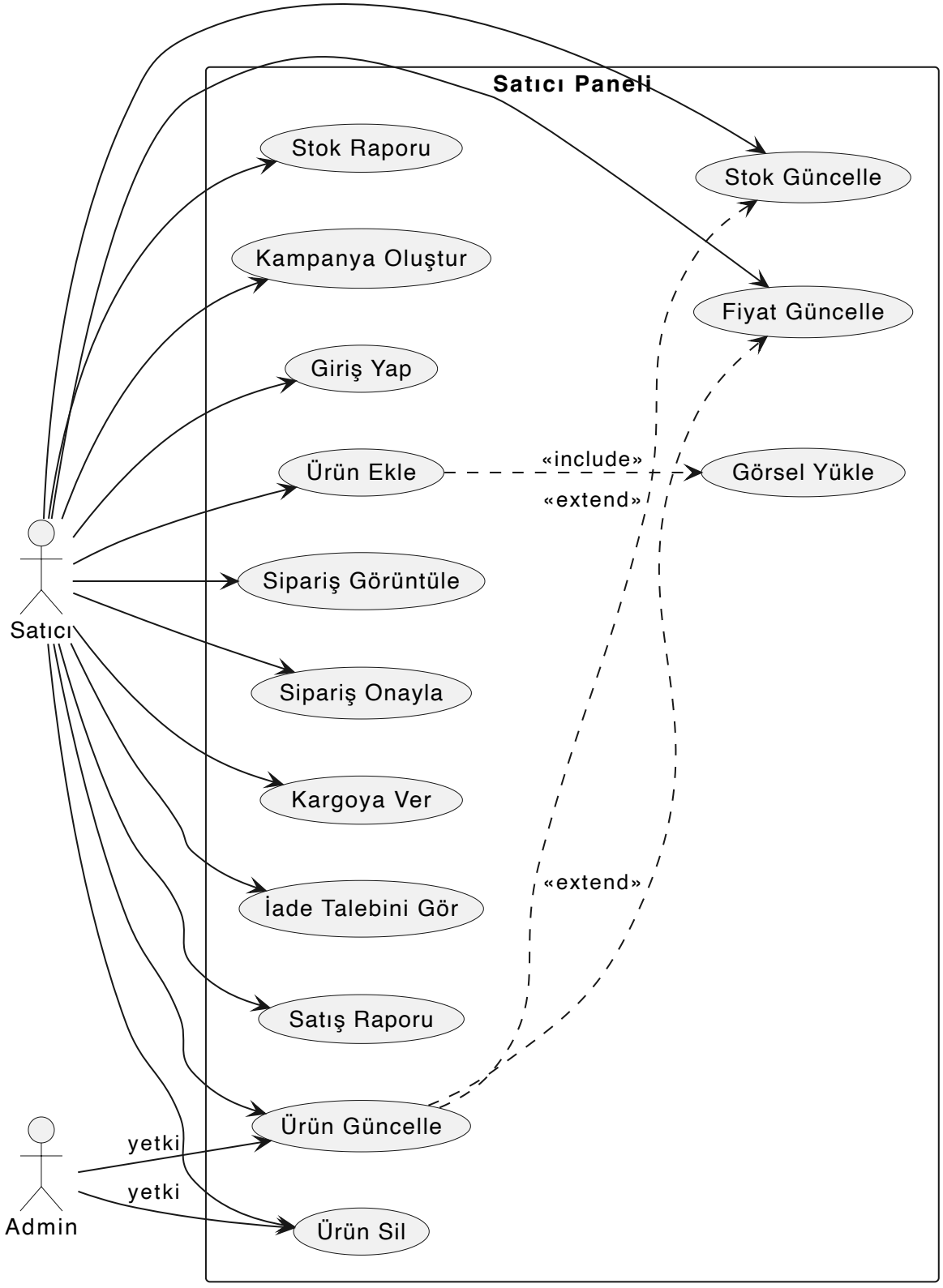
Mermaid

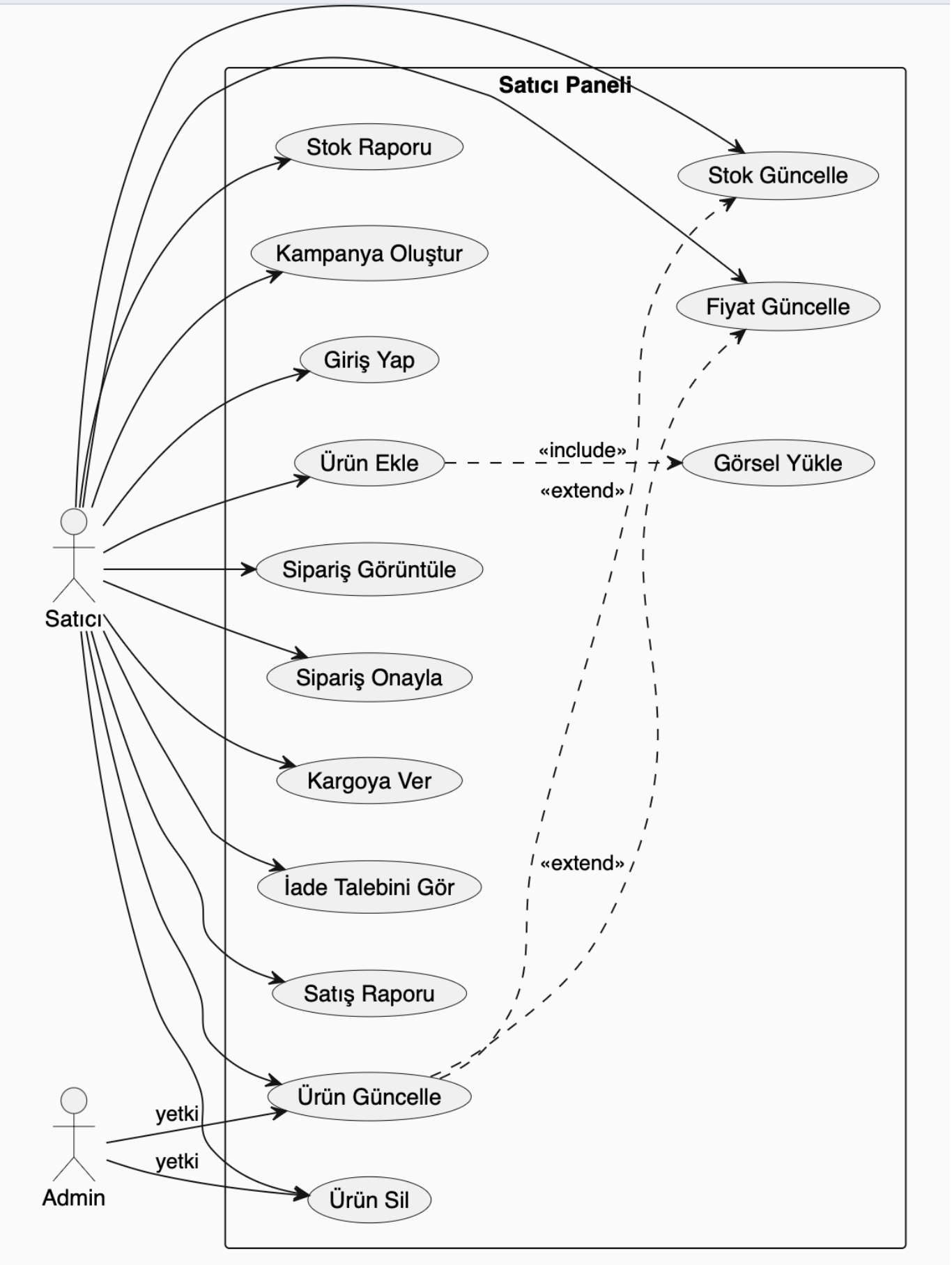




3. SATICI (SELLER) USE CASE DİYAGRAMI

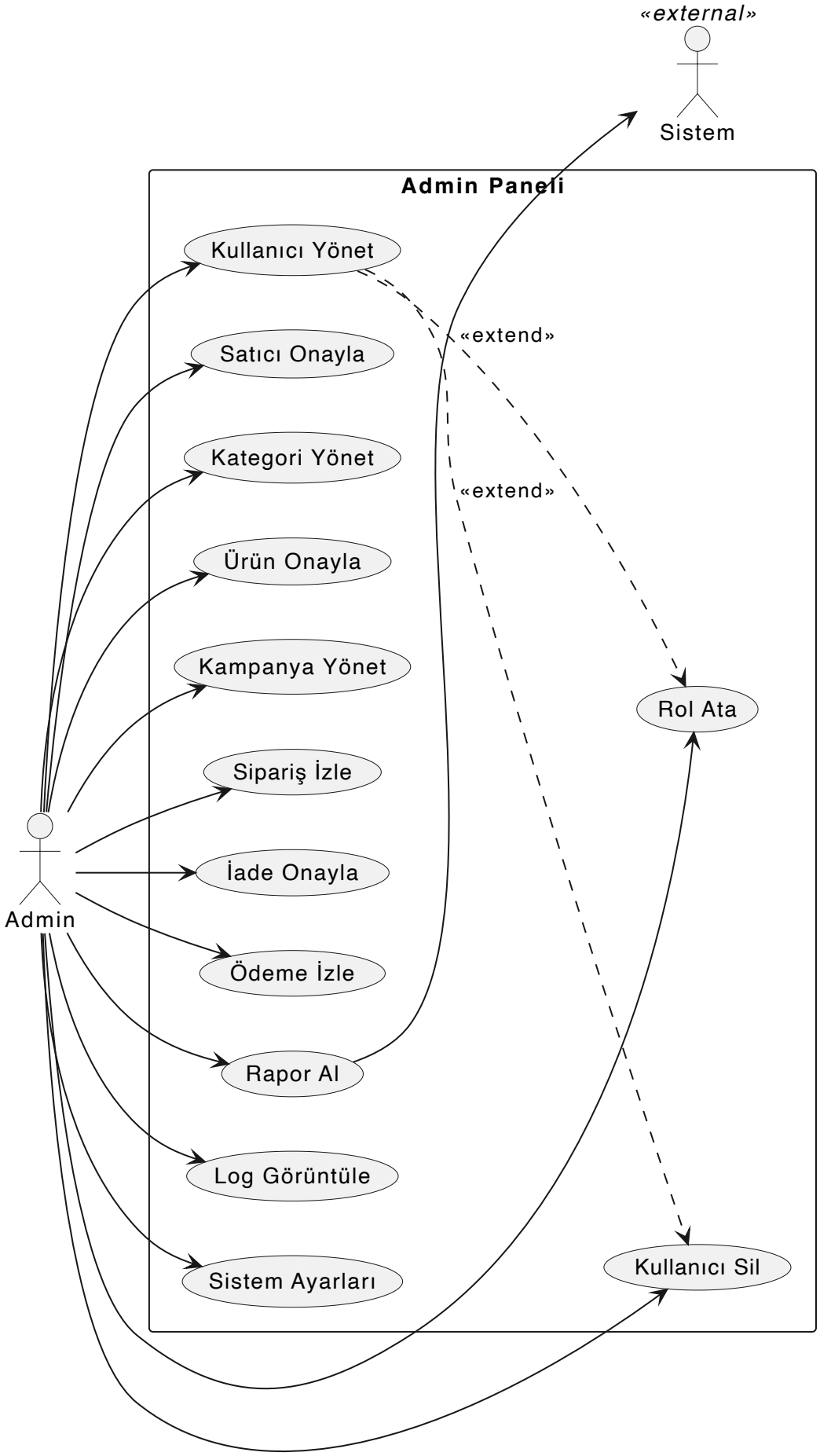
PlantUML

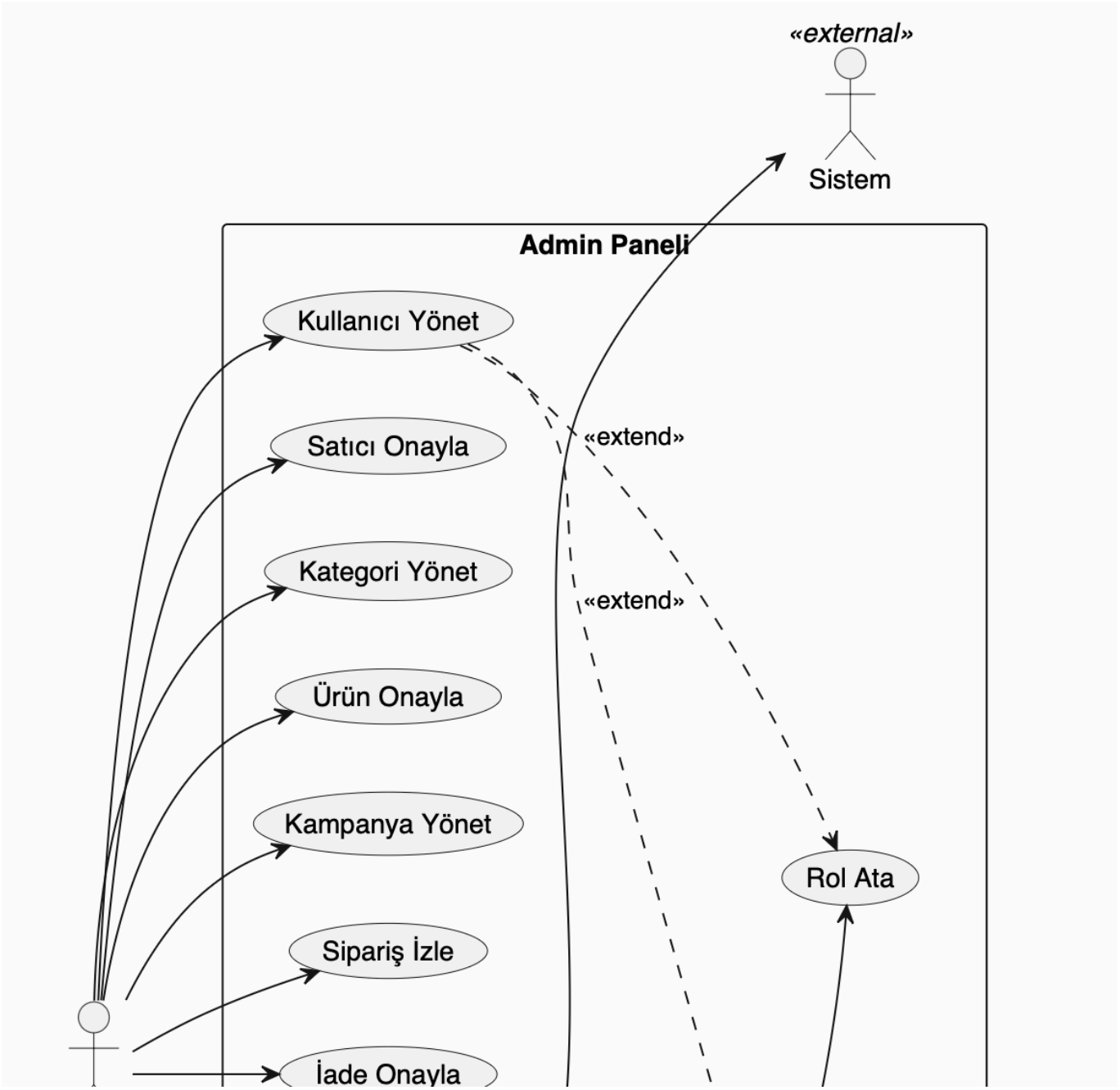

Satıcı Paneli
Stok Raporu
Stok Güncelle
Kampanya Oluştur
Fiyat Güncelle
Giriş Yap
Ürün Ekle
«include»
Görsel Yükle
«extend»
Satıcı
Sipariş Görüntüle
Sipariş Onayla
Kargoya Ver
«extend»
İade Talebini Gör
Satış Raporu
Ürün Güncelle
yetki
yetki
Admin
Ürün Sil

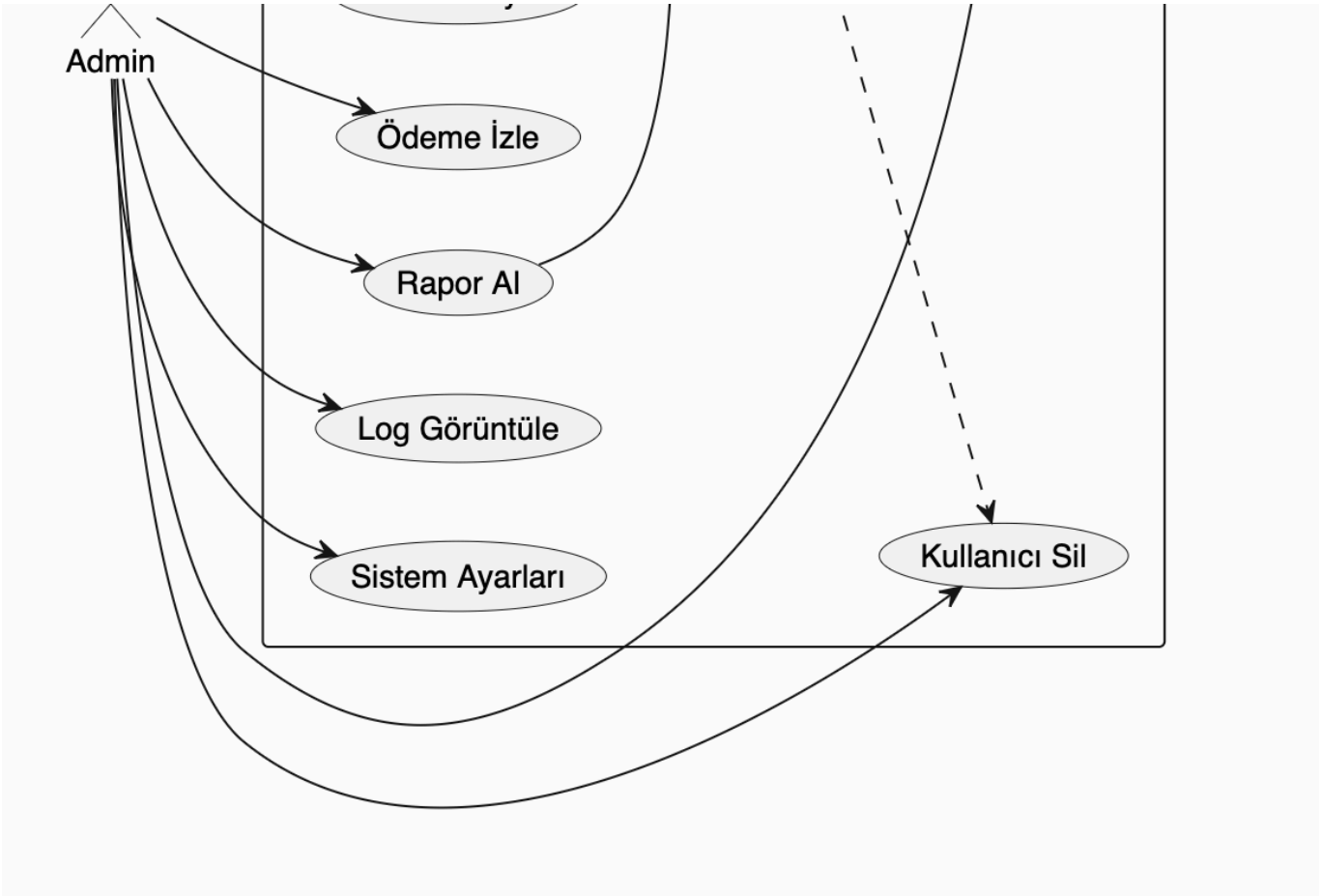


4. ADMIN USE CASE DİYAGRAMI

PlantUML


«external»
Sistem
Admin Paneli
Kullanıcı Yönet
Satıcı Onayla
Kategori Yönet
Ürün Onayla
Kampanya Yönet
Sipariş İzle
İade Onayla
Ödeme İzle
Rapor Al
Log Görüntüle
Sistem Ayarları
«extend»
«extend»
Rol Ata
Kullanıcı Sil
Admin

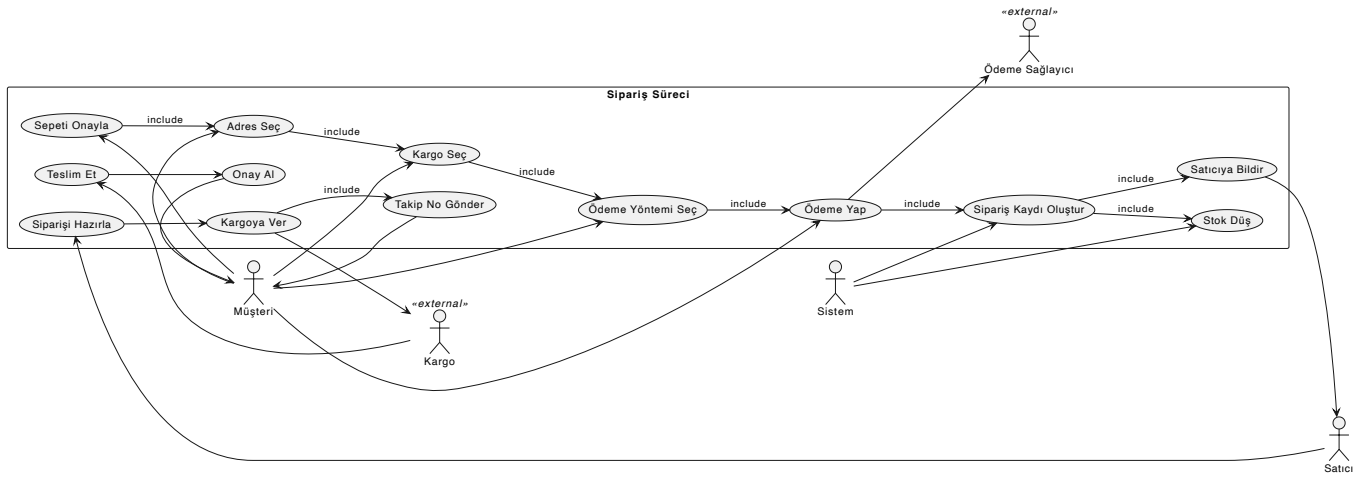

«external»
Sistem
Admin Paneli
Kullanıcı Yönet
Satıcı Onayla
Kategori Yönet
Ürün Onayla
Kampanya Yönet
Sipariş İzle
İade Onayla
«extend»
«extend»
Rol Ata

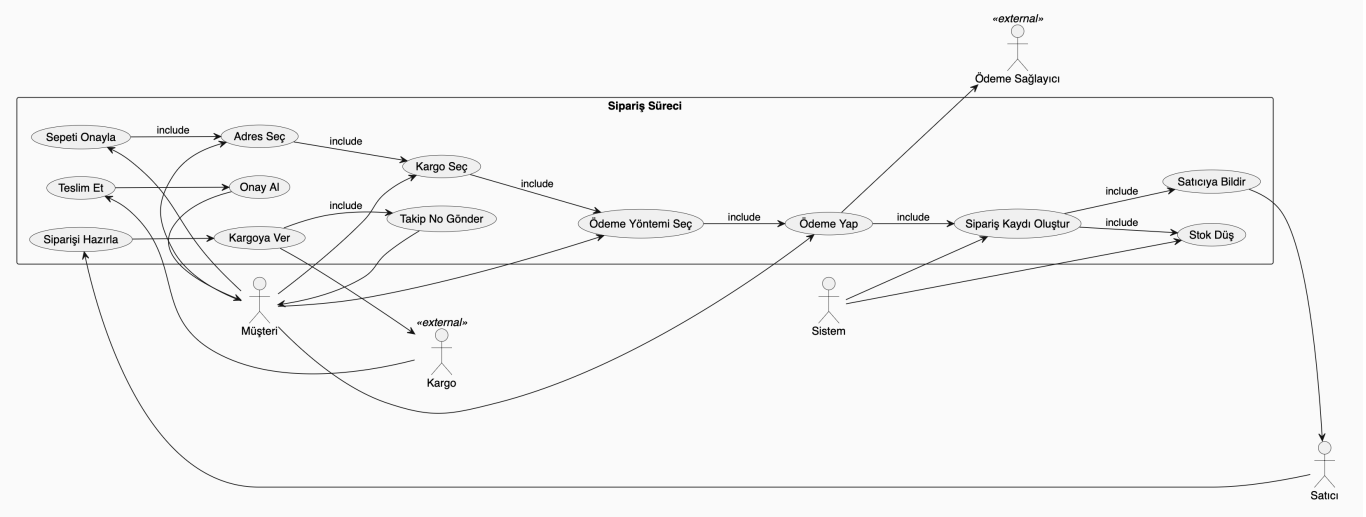


5. SİPARİŞ SÜRECİ USE CASE DİYAGRAMI (Detaylı)

Sipariş akışını adım adım gösterir.

PlantUML





6. AKTÖR-USE CASE İLİŞKİ MATRİSİ

Use Case	Misafir	Müşteri	Satıcı	Admin
Üye Ol	✅	–	–	–
Giriş Yap	–	✅	✅	✅
Ürün Ara	✅	✅	✅	✅
Ürün İncele	✅	✅	✅	✅
Sepete Ekle	–	✅	–	–
Sipariş Ver	–	✅	–	–
Ödeme Yap	–	✅	–	–
Sipariş Takip	–	✅	✅	✅
İade Talebi	–	✅	✅	✅
Ürün Ekle	–	–	✅	✅
Stok Güncelle	–	–	✅	✅
Kargoya Ver	–	–	✅	–
Kullanıcı Yönet	–	–	–	✅
Satıcı Onayla	–	–	–	✅
Rapor Al	–	–	✅	✅
Sistem Ayarları	–	–	–	✅

---

7. USE CASE AÇIKLAMALARI (Örnekler)

UC-06: Sipariş Ver

Alan	Değer
Aktör	Müşteri
Ön Koşul	Giriş yapılmış, sepette ürün var
Ana Akış	1. Sepeti onayla → 2. Adres seç → 3. Kargo seç → 4. Ödeme yöntemi seç → 5. Ödeme yap → 6. Sipariş oluştur
Alternatif	Ödeme başarısız → hata mesajı
Son Koşul	Sipariş kaydı oluşturulur, e-posta gönderilir
Include	Ödeme Yap, Stok Düş
Extend	Kupon Uygula

UC-15: Ödeme Yap

Alan	Değer
Aktör	Müşteri, Ödeme Sistemi
Ön Koşul	Sipariş oluşturulmuş
Ana Akış	1. Kart bilgisi gir → 2. 3D Secure → 3. Onay al → 4. Sonuç dön
Alternatif	Kart reddi, zaman aşımı
Son Koşul	Ödeme kaydı, sipariş "hazırlanıyor"

UC-18: İade Talebi

Alan	Değer
Aktör	Müşteri
Ön Koşul	Teslim edilmiş sipariş, iade süresi içinde
Ana Akış	1. Sipariş seç → 2. Ürün seç → 3. Sebep gir → 4. Talep gönder
Alternatif	Süre dışı → reddedilir
Son Koşul	İade kaydı, satıcıya bildirim

---

8. ÖZET

Diyagram	Amaç
Genel Sistem	Tüm aktör ve use case'ler
Müşteri	Alışveriş yolculuğu
Satıcı	Ürün/sipariş yönetimi

Diyagram	Amaç
Admin	Sistem yönetimi
Sipariş Süreci	Uçtan uca akış
Matris	Yetki kontrolü